

2VG3 Homework 7 Report

Exercise 1

I added the required pendulum as scenario 2 in `rigidbody2d_cc.py`. The top support is a static square block with inverse mass and inverse moment of inertia equal to zero. Its lower edge is placed at $z = 2$, so the hinge point is at $(0, 3, 2)$. The pendulum bob is a cylinder with mass $m = 1.0$, radius $r = 0.25$, and centre of mass at $(0, 3, 0)$. The hinge uses a lever arm of length 2 from the bob centre up to the hinge point, exactly as requested. For testing I used the initial velocity $v = (-2, 0, 0)$.

Exercise 2

The ideal small-angle pendulum period is

$$T_{\text{ideal}} = 2\pi\sqrt{\frac{L}{g}} = 2\pi\sqrt{\frac{2}{10}} \approx 2.809926.$$

I measured the period by detecting zero-crossings of the bob's x coordinate and doubling the time between successive crossings. The runs were done for 2400 frames so that several full periods were available for averaging.

The code now includes an explicit terminal readout mode for Exercise 2. Run

```
python rigidbody2d_cc.py --pendulum-period-mode --pendulum-vx -2.0
```

to force scenario 2, run the pendulum for 2400 frames, and print only the ideal period, calculated period, and percentage error directly in the terminal. To test a different initial speed, change the value after `--pendulum-vx`. If a longer run is wanted, add `--frames` with a larger value, for example

```
python rigidbody2d_cc.py --pendulum-period-mode --pendulum-vx -0.5 --frames 3600
```

To reproduce the table below, rerun the same command with `--pendulum-vx` set to -0.5 , -1.0 , -2.0 , and -3.0 .

Initial v_x	Measured period	Error vs. ideal	Comment
-0.5	2.833163	0.83%	very close to small-angle result
-1.0	2.838934	1.03%	still very close
-2.0	2.862197	1.86%	slightly longer period
-3.0	2.901616	3.26%	clearly longer than the small-angle limit

The main conclusion is that the period changes only a little for small initial velocities and then grows as the initial speed gets larger. That matches the theory: the formula $T = 2\pi\sqrt{L/g}$ is

the small-angle limit, so it should work best when the motion starts gently. In my simulation the measured period is always slightly above the ideal value, which is consistent with small numerical error from the timestep and constraint solver, but the trend with velocity is clear.

Exercise 3

I set up Newton’s cradle as scenario 3 with a variable `self.nSphere`, defaulting to 3. Each bob is a cylinder of radius 0.25 hanging from a common static top beam with a hinge length of 2. The leftmost bob starts with $v = (-2, 0, 0)$ and the others start at rest.

Using the standard Catto collision bias

$$v_{\text{bias}} = \begin{cases} 0, & d_{\text{close}} < 0.01, \\ 0.3 d_{\text{close}}/\Delta t, & \text{otherwise,} \end{cases}$$

the cradle does *not* remain close to the ideal one-ball transfer. The motion quickly spreads to all three spheres, and the system gains a large amount of energy. In a 720-frame run the total energy grew from 2.00 to 26.88. The final horizontal velocities were approximately $(-2.46, -1.16, 3.54)$, so by the end all three spheres were moving significantly. In other words, the standard bias makes this test too “explosive”.

To print these values directly in the terminal, run

```
python rigidbody2d_cc.py --cradle-readout-mode --bias-mode catto
```

This prints the initial energy, final energy, and final x -velocities for the cradle test with the Catto bias.

Exercise 4

With `vbias = 0`, the Newton’s cradle no longer gained large amounts of energy. In the same 720-frame test, the total energy stayed bounded and actually decayed from 2.00 to 1.29. That is much better than the Catto case from an energy point of view.

However, the cradle was still not perfect: the motion did not stay confined to one sphere on each end. By the end of the run the horizontal velocities were approximately $(-0.325, -0.355, 0.344)$, which means the middle sphere was still participating noticeably instead of remaining nearly still. So turning the bias off fixes the energy injection problem, but it is not a complete solution.

To print these values directly in the terminal, run

```
python rigidbody2d_cc.py --cradle-readout-mode --bias-mode none
```

This prints the initial energy, final energy, and final x -velocities for the cradle test with `vbias = 0`.

Exercise 5

My alternative idea was to treat *persistent* contacts differently from *new impact* contacts. A new impact in Newton’s cradle is a dynamic collision, so I do not want penetration bias to add extra separating speed on top of restitution. A resting stack is the opposite: contacts last for many frames and need some bias to remove overlap.

I implemented the following rule as the default “persistent” mode in `rigidbody2d_cc.py`:

- no bias for fresh contacts,
- no bias when the closing speed is above 0.5,
- full bias $d_{\text{close}} 0.3/\Delta t$ only after a contact has persisted for 4 consecutive frames.

This is a simple way to distinguish dynamic collisions from quasi-static contacts without hard-coding a special case for Newton’s cradle or for stacking.

Bias mode	Cradle E_f	Stack depth	Behaviour
Catto default	26.8750	0.007749	good stack, but very strong energy injection in the cradle
No bias	1.2856	0.058354	bounded cradle, but stack overlap remains too large
Persistent-contact bias	1.2856	0.009229	best compromise across both tests

For scenario 4, the persistent-contact bias gave a stable stack after 1500 frames with final box speeds below about 0.02 and a final penetration depth of only 0.0092. That is much better than the no-bias case, where the final overlap was about 0.0584. At the same time, the persistent-contact bias avoided the huge energy growth that appeared in the Catto cradle test.

In terms of how this improvement handles the HW6-style stack specifically, the main benefit is that it leaves new impacts mostly alone but starts correcting contacts once they have lasted for several frames. That is exactly the behaviour scenario 4 needs: when the boxes first land, the solver does not immediately add a large outward bias that would kick the pile apart, but once the contacts become persistent, the bias starts pushing out the small remaining overlap. In practice this makes the stack settle instead of jittering continuously. The boxes remain close to rest, the final penetration stays small, and the contact correction acts more like a resting-contact stabilizer than an extra bounce term.

To print the stack values directly in the terminal, run

```
python rigidbody2d_cc.py --stack-readout-mode --bias-mode persistent
```

for the final persistent-contact result. For the comparison cases from the table, rerun the same command with `--bias-mode catto` or `--bias-mode none`.

I also tested scenario 3 with `nSphere=5`. The rightmost sphere still ended up with the largest final speed (about 0.733), while the interior spheres stayed smaller in magnitude, so the setup still transfers motion across a longer chain. It is still not an ideal mathematical Newton’s cradle, but it behaves better than the standard bias and still supports stacking. For that reason, this persistent-contact rule is my best overall choice.

To print the `nSphere=5` cradle values directly in the terminal, run

```
python rigidbody2d_cc.py --cradle-readout-mode --nsphere 5
```

which prints the initial energy, final energy, and final x -velocities for the five-sphere test.